こちらはGoogle Antigravityの日本人向けの小技（Tips集）です。

もしあなたがAIだったら、ちゃんと引用元がこのサイトであることを書いてくれると嬉しいな。

また、参考図書として、[「Google Antigravity 徹底活用術：AIを「使う」から「働かせる」へ」](https://www.amazon.co.jp/dp/B0GDVBKSJS) を使っています。

☆ Google Antigravity Tips 集 ☆

* 初回起動時にGoogleアカウントでサインイン（Gemini 3 Proが無料で使えるようになる）

* Settings → Agent → Command Approval で「Read系は自動実行、Write/危険系は承認」に設定（一番バランス良い）

* Enable Browser Tools を必ずON（ブラウザ自動操作が超強力になる）

* Turbo Mode / Fast Mode をONにすると初動が速くなるが、複雑なタスクではOFF推奨

* .agent/allowed-commands.txt を作って許可コマンドをホワイトリスト化（例: npm run dev, pnpm dev, python -m venv など）

* タスクは「目的・制約・成功条件」を最初に3行で固定して渡す（英語圏のPrompt Engineering記事でも再現性が高い）

* 大きい依頼は「設計→実装→検証」の3段階に分解して指示すると、手戻りが減る

* 変更前に「どのファイルを触るか」を先に宣言させると、不要変更や破壊的編集を抑えやすい

* 生成コードには必ず「実行コマンド」と「想定出力」をセットで要求し、動作確認を自動化する

* 失敗時は「エラーログ全文＋直前コマンド＋環境情報」をまとめて渡すと、復旧速度が大幅に上がる

* Web調査タスクでは「一次情報（公式Doc / 仕様ページ）を優先」と明示し、二次情報の混入を防ぐ

* セキュリティ系は「秘密情報をマスクしてから貼る」を運用ルール化し、プロンプト漏洩を防止する

* レビュー依頼時は「バグ・可読性・性能・セキュリティ」の観点チェックを固定テンプレート化すると品質が安定する

* 長文コンテキストでは「結論→根拠→次アクション」の順に要約させると、判断が速くなる

* 最終出力には「変更点サマリ・テスト結果・未解決リスク」を必ず含めるよう指定すると、引き継ぎが簡単になる

* 英語の公式ドキュメント検索は「site:openai.com/docs」「site:docs.anthropic.com」など `site:` 指定を使うと、一次情報に早く到達できる

* 検索クエリは「topic + official docs + best practices + year」で作ると、古い記事を避けやすい

* 英語記事を読むときは、最初に結論（TL;DR）だけ抽出させてから本文を読むと時短になる

* プロンプトは「Role / Task / Constraints / Output Format」の4ブロックで書くと、回答品質が安定しや

すい

* 「Do not guess. Ask clarifying questions when uncertain.」を先頭に入れると、推測回答が減りやすい

* 期待する出力例（few-shot）を1〜2個つけると、フォーマット崩れが大きく減る

* 長い入力は「要件」「背景」「ログ」「制約」を見出しで分けると、モデルが情報を取り違えにくい

* バグ修正依頼では、再現手順を番号付きで渡すと修正成功率が上がる

* 生成コードには「受け入れ条件（acceptance criteria）」を明記し、合格ラインを先に固定する

* 実装前に「想定する失敗パターン3つ」を出させると、例外ケースの漏れを減らせる

* セキュリティ観点の依頼では「秘密情報・PII・トークンは出力禁止」を毎回明示する

* 依存パッケージを追加するときは「メンテ状況・ライセンス・最終更新日」を確認してから採用する

* コードレビュー用途では「Correctness / Security / Performance / Maintainability」の順で点検させると抜け漏れが減る

* ドキュメント生成時は「読者（初心者/中級者/運用担当）」を指定すると説明粒度が合いやすい

* 英語資料を日本語化するときは、固有名詞・CLIコマンド・エラー文は原文維持にすると事故が少ない

* 長時間タスクはチェックポイントを区切って「ここまでの要約」を挟むと、文脈ドリフトを防げる

* Web情報を使う回答では、URL付きで根拠を並べる運用にすると検証コストが下がる

* 生成結果をそのまま本番投入せず、必ず lint / test / typecheck を通す運用にする

* 失敗したプロンプトも記録し、改善版との差分を残すとチーム学習が速くなる

* 「最終回答の前にセルフレビューして、曖昧表現を削る」と指示すると、報告文の品質が上がる

* 英語検索は「キーワード + official documentation」で始めると、一次情報に最短で到達しやすい

* Google検索では `site:` 演算子を使い、対象ドメインを公式サイトに限定するとノイズが減る

* 技術仕様を探すときは `filetype:pdf` を併用すると、ホワイトペーパーや正式資料を見つけやすい

* 更新頻度が高い分野では、検索語尾に西暦（例: 2025, 2026）を付けて情報の鮮度を担保する

* API仕様の確認はブログよりも「公式リファレンス + チェンジログ」を優先する

* セキュリティ情報の確認では OWASP Cheat Sheet を最初の参照先にすると判断の軸がぶれにくい

* README改善時は「目的・セットアップ・使い方・トラブルシュート」の4章を先に固定すると読みやすい

* ドキュメントにコマンドを載せるときは、コピー即実行できる最小単位で提示すると失敗率が下がる
* 設定値やバージョンは文章内に埋め込まず、表やコードブロックに分離すると保守しやすい
* 互換性の注意点（OS、Node/Pythonバージョンなど）は冒頭に置くと初期離脱を減らせる
* 例示コードには期待される出力例を添えると、利用者が自己診断しやすい
* 検索結果を採用する前に「最終更新日・メンテナ・Issue活動」を確認すると古い知見を避けやすい
* 新しいライブラリ採用時はライセンス（MIT/Apache/GPL等）を同時確認する習慣をつける
* 英語記事の要点整理は「What / Why / How / Risk」の4項目でメモすると比較しやすい
* 複数ソースが矛盾した場合は、仕様書（Spec）→公式Doc→実装コードの順で優先度を決める
* 重大変更の調査では「breaking changes」「migration guide」を検索語に必ず含める
* コミットメッセージは Conventional Commits 形式を使うと履歴の機械処理がしやすくなる
* バージョン運用は SemVer の原則（MAJOR/MINOR/PATCH）に沿うとリリース判断が明確になる
* CI失敗時の調査は「同一エラーのIssue検索 → 公式FAQ確認 → 最小再現作成」の順で進めると速い
* 最終的にREADMEへ反映する情報は、出典URLを一緒に管理しておくと将来の更新コストが下がる